Reflection

Classpath, Casting, Annotations

Yegor Bugayenko

Lecture #7 out of 8
80 minutes

The slidedeck was presented by the author in this YouTube Video.

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

What Is Reflection?

Type Casting and Subsumption

Factory Method

Classpath Scanning

Annotations

Discrimination by Type

Chapter #1:

What Is Reflection?

Simple Example of Reflection in Java

```
1 void print(Object x) {
2   if (x instanceof File)
    {
3     System.out.println("
    Yes!");
4   }
5 }
```

"Reflective computational systems allow computations to observe and modify properties of their own behavior. It would be desirable for computations to avail themselves of these reflective capabilities, examining themselves in order to make use of meta-level information in decisions about what to do next."

Source: Jonathan M. Sobel and Daniel P. Friedman. *An Introduction to Reflection-Oriented Programming*. Citeseer, 1996



Yue Li

"As reflection is increasingly used in Java programs, the cost of imprecise reflection handling has increased dramatically... Almost all the analyses reported are unsound in the presence of reflection since it is either ignored or handled partially."

— Yue Li et al. "Self-Inferencing Reflection Resolution for Java". In: *Proceedings of the 28th European Conference on Object-Oriented Programming*. Springer. 2014, pp. 27–53. DOI: 10.1007/978-3-662-44202-9_2

Chapter #2:

Type Casting and Subsumption

Iterable → Collection

Downcasting (wrong!):

```
1  int sizeOf(Iterable items) {
2    int size = 0;
3    if (items instanceof Collection)
     {
4      size = ((Collection) items).
     size();
5    } else {
6      for (Object item : items) {
7        ++size;
8      }
9    }
10   return size;
11 }
```

Method overloading (right!):

```
1  int sizeOf(Iterable items) {
2    int size = 0;
3    for (Object item : items) {
4      ++size;
5    }
6    return size;
7  }
8
9  int sizeOf(Collection items) {
10   return items.size();
11 }
```

[Iterable Coupling JEP-394 Matching]

Implicit Coupling

```
int sizeOf(Iterable items) {
  int size = 0;
  if (items instanceof Collection)
  {
    size = ((Collection) items).
  size();
  } else {
    for (Object item : items) {
      ++size;
    }
  }
  return size;
}
```

```
Iterable<Book> books1 =
  new PgBooks1("localhost:5432");
int s1 = sizeOf(books1);

Collection<Book> books2 =
  new PgBooks2("localhost:5432");
int s2 = sizeOf(books2);
```

It may be hard to understand why —s2— is evaluated much faster than —s1—, while the signature of —sizeOf()— is the same [1].

[Iterable Coupling JEP-394 Matching]

Pattern Matching in Java 16

Java 11 (wrong!):

```
1  int sizeOf(Iterable items) {
2    int size = 0;
3    if (items instanceof Collection)
     {
4      size = ((Collection) items).
     size();
5    } else {
6      for (Object item : items) {
7        ++size;
8      }
9    }
10   return size;
11 }
```

Java 16 (**even worse!**):




```
1  int sizeOf(Iterable items) {
2    int size = 0;
3    if (items instanceof Collection
     c) {
4      size = c.size();
5    } else {
6      for (Object item : items) {
7        ++size;
8      }
9    }
10   return size;
11 }
```

[Iterable Coupling JEP-394 Matching]

[Iterable Coupling JEP-394 Matching]

C#, Rust, and Pattern Matching

C#:

```
public int sizeOf<T>(IEnumerable<T> items) {
  if (items is IList<T> list) {
    return list.Count;
  } else {
    return // count them one by one
  }
}
```

Some other languages have pattern matching feature, including Kotlin, Scala, Haskell, Elixir, Swift, F#, and Erlang, ... which contradicts the principle of encapsulation.

Rust:

```
enum Color {
  RGB(u8, u8, u8),
  Transparent
}
fn paint(c: Color) {
  match c {
    Color::RGB(r, g, b) =>
      println!("#{r}{g}{b}"),
    Color::Transparent =>
      println!("none")
  }
}
fn main() {
  let c = Color::RGB(64, 16, 0);
  paint(c);
}
```

Chapter #3:

Factory Method

[IF forName]

Conditional Object Construction

This is wrong:

```
interface Figure
  int area();
class Square implements Figure
class Triangle implements Figure
class Polygon implements Figure

class FactoryOfFigures
  Figure make(int sides) {
    if (sides == 3) {
      return new Triangle();
    } else if (sides == 4) {
      return new Square();
    } else {
      return new Polygon(sides);
    }
  }
```

This is better:

```
class PolymorphicFigure implements Figure
  PolymorphicFigure(int sides)
  @Override int area() {
    if (sides == 3) {
      return new Triangle().area();
    } else if (sides == 4) {
      return new Square().area();
    } else {
      return new Polygon(sides).area();
    }
  }
```

Here, the semantic of object construction is not visible to the client—the coupling is “loose” [5].

[IF forName]

Generating class name from a string

This is wrong:

```
1  interface Figure
2    int area();
3
4  class Square implements Figure
5  class Triangle implements Figure
6  class Polygon implements Figure
7
8  class FactoryOfFigures
9    Figure make(String name) throws Exception {
10     Class<?> c = Class.forName(name);
11     return c.getConstructor().newInstance();
12   }
```

This is better:

```
1  class PolymorphicFigure implements Figure
2    PolymorphicFigure(String name)
3    @Override int area() {
4      if (name.equals("Triangle")) {
5        return new Triangle().area();
6      } else if (name.equals("Square")) {
7        return new Square().area();
8      } else {
9        return new Polygon(?).area();
10     }
11   }
```

This is better since the mechanics of class finding is explicit—no surprises expected [2].

Chapter #4:

Classpath Scanning

Finding Java Classes

```
interface Foo {}

class Bar implements Foo {}

Reflections rts =
  new Reflections("");
Set<Class<?>> types = rts.get(
  SubTypes.of(Foo.class).asClass()
);
```

```
public @interface Foo {}

@Foo
class Bar {}

Reflections rts =
  new Reflections("");
Set<Class<?>> types = rts.get(
  SubTypes.of(
    TypesAnnotated.with(Foo.class)
  ).asClass()
);
```

[Class]

The library is called Reflections. Instead, use explicit object instantiation.

Chapter #5:

Annotations

In lieu of static methods

```
interface Pub
  String isbn();

class Book implements Pub
  @Override public String isbn()
    /* ... */
  public static String category()
    return "book";

class Journal implements Pub
  @Override public String isbn()
    /* ... */
  public static String category()
    return "journal";
```

```
interface Pub
  String isbn();

@Target(ElementType.CLASS)
@Retention(RetentionPolicy.SOURCE)
public @interface Category
  String value();

@Category("book")
class Book implements Pub
  @Override public String isbn()
    /* ... */

@Category("journal")
class Journal implements Pub
  @Override public String isbn()
    /* ... */
```

Locating Methods

```
1  @Target(ElementType.METHOD)
2  @Retention(RetentionPolicy.SOURCE)
3  public @interface Path
4    String url;
5
6  class BookController
7    @Path("/book-title")
8    String title()
9      // Build HTML page and return
     it
```

```
1  String dispatch(String url) {
2    c = new BooksController();
3    b = BooksController.class;
4    for (Method m : b.getDeclaredMethods()) {
5      if (m.isAnnotationPresent(Path.class)) {
6        Annotation a = m.getAnnotation(Path.class);
7        if (a.url().equals(url)) {
8          return m.invoke(c);
9        }
10     }
11   }
12   return "404 Page not found";
13 }
```

The —BookController— doesn't know how exactly the data in its annotation is being used and by whom [4].

Dependency Injection Container

```
1  interface Shipment
2    int cost();
3
4  class Cart
5    @Inject private Shipment shmt;
6    private Book book;
7    void setBook(Book b)
8      this.book = b;
9    int cost()
10     return this.book.price() + this.shmt.cost();
11
12 container = new Container();
13 c = container.make(Cart.class);
14 c.setBook(new Book("1984"));
15 x = c.cost();
```

```
1  class Container {
2    private HashMap<Class, Object> cache =
3      new ConcurrentHashMap<>();
4    T make(Class<T> type) {
5      // 1. Find @Inject-annotated "shmt" field;
6      // 2.  Make an instance of "Shipment";
7      // 3. Store it in the "cache";
8      // 4. Make an instance of "Cart";
9      // 5. Store "cart" in the "cache";
10     // 6. Assign "shipment" to "cart.shmt";
11     // 7. Return "cart".
12   }
13 }
```

How do you think, at the step no.2, what class will be instantiated? [3]

Dependency Injection *without* a Container

```
1  interface Shipment
2    int cost();
3
4  class Cart
5    @Inject private Shipment shmt;
6    private Book book;
7    void setBook(Book b)
8      this.book = b;
9    int cost()
10     return this.book.price() + this.shmt.cost();
11
12 container = new Container();
13 c = container.make(Cart.class);
14 c.setBook(new Book("1984"));
15 x = c.cost();
```

```
1  interface Shipment
2    int cost();
3
4  class Cart
5    private final Shipment shmt;
6    private final Book book;
7    Cart(Shipment s, Book b)
8      this.shmt = s;
9      this.book = b;
10   int cost()
11     return this.book.price() + this.shmt.cost();
12
13 c = new Cart(new MyShipment(), new Book("1984"));
14 x = c.cost();
```

Chapter #6:

Discrimination by Type

Polymorphism vs. Casting

```
1 interface Figure
2   void rotate(int d);
3 class Circle implements Figure
4   void rotate(int d) //...
5   int radius() //...
6 class Square implements Figure
7   void rotate(int d) //...
8   int side() //...
```

Philosophically speaking, type casting is discrimination [6].

```
1  // This is polymorphism:
2  int area(Figure f)
3    return f.area();
4
5  // This is type casting:
6  int area(Figure f)
7    if (f instanceof Circle c) {
8      return c.radius()
9    } else if (f instanceof Square s)
     {
10     return s.side() * s.side();
11   } else {
12     throw new Exception("oops");
13   }
```

[Polymorphism]

References

[1] Yegor Bugayenko. *Class Casting Is a Discriminating Anti-Pattern*. https://www.yegor256.com/150402.html. [Online; accessed 22-09-2024]. Apr. 2015.

[2] Yegor Bugayenko. *Constructors or Static Factory Methods?* https://www.yegor256.com/171114.html. [Online; accessed 22-09-2024]. Nov. 2017.

[3] Yegor Bugayenko. *Dependency Injection Containers Are Code Polluters*. https://www.yegor256.com/141003.html. [Online; accessed 22-09-2024]. Jan. 2014.

[4] Yegor Bugayenko. *Java Annotations Are a Big Mistake*. https://www.yegor256.com/160412.html. [Online; accessed 22-09-2024]. Apr. 2016.

[5] Yegor Bugayenko. *Reflection Means Hidden Coupling*. https://www.yegor256.com/220605.html. [Online; accessed 22-09-2024]. June 2022.

[6] Yegor Bugayenko. *Strong Typing Without Types*. https://www.yegor256.com/201110.html. [Online; accessed 22-09-2024]. Nov. 2020.

[7] Yue Li et al. "Self-Inferencing Reflection Resolution for Java". In: *Proceedings of the 28th European Conference on Object-Oriented Programming*. Springer. 2014, pp. 27–53. DOI: 10.1007/978-3-662-44202-9_2.

[8] Jonathan M. Sobel and Daniel P. Friedman. *An Introduction to Reflection-Oriented Programming*. Citeseer, 1996.